

3.2 Regresión polinomial

M. Sc. Huáscar Fedor Gonzales Guzmán · Carrera de Ingeniería Informática · UATF · Gestión 2026-02

La teoría necesaria para comprender el notebook [regresion_polinomial.ipynb](#): modelo, mínimos cuadrados, escalamiento, métricas, sesgo-varianza, validación cruzada, residuos, interpretación y regularización.

Cómo usar estos apuntes

Estos apuntes acompañan al notebook [regresion_polinomial.ipynb](#). El notebook muestra **qué se hace y cómo se programa**; aquí se explica **por qué funciona**. La recomendación es leer cada sección de los apuntes antes de ejecutar la sección correspondiente del notebook y volver a ellos cuando un resultado no quede claro.

Todos los números citados (coeficientes, RMSE, correlaciones) provienen del mismo experimento del notebook: 60 observaciones sintéticas generadas con la semilla 32, divididas en 45 de entrenamiento y 15 de prueba.

Sección del notebook	Sección de estos apuntes
1–2. Generación y exploración de datos	1. El problema: relaciones que no son rectas
3. División entrenamiento / prueba	7. Validación cruzada y selección del grado
4. Modelo base lineal y sus residuos	1, 3 y 8
5. La transformación polinomial por dentro	2. El modelo · 4. Escalas y multicolinealidad
6. Regresión polinomial con scikit-learn (Pipeline)	3. Mínimos cuadrados · 4. Estandarización
7. Efecto del grado	6. Sesgo, varianza y el grado del polinomio
8. Selección del grado con validación cruzada	7. Validación cruzada y selección del grado
9. Evaluación final en prueba	5. Métricas de evaluación
10. Análisis de residuos	8. Análisis de residuos
11. Velocidad óptima	9. Interpretación mediante derivadas
12. Extrapolación	10. El peligro de extrapolar
13. Conclusiones (Ridge)	11. Regularización Ridge · 12. Resumen

Símbolo	Significado
x, v	variable de entrada (predictor); en el ejemplo, la velocidad en km/h
y	variable de salida (respuesta); el consumo en L/100 km
$\hat{y}$	predicción del modelo ("y sombrero")

Símbolo	Significado
n, d	número de observaciones y grado del polinomio
β_j, β^*	coeficientes (parámetros) del modelo; β^* son los valores estimados
$h(x)$	modelo ajustado (hipótesis) que aproxima a la función verdadera $f(x)$
$e_i = y_i - \hat{y}_i$	residuo de la observación i
ϵ, σ	ruido aleatorio y su desviación estándar (en el ejemplo $\sigma = 0.5$)
X, X^T	matriz de diseño y su transpuesta

01 El problema: relaciones que no son rectas

La **regresión** es la tarea de aprendizaje supervisado que predice una variable **numérica continua** y a partir de una o más variables de entrada x . El caso más sencillo, la regresión lineal simple, supone que la relación es una recta. Muchos fenómenos reales, sin embargo, siguen una **curva**.

1.1 El modelo que genera los datos

Todo problema de regresión parte de suponer que los datos observados se producen así:

$$y = f(x) + \epsilon, \quad \epsilon \sim N(0, \sigma^2) \quad (1.1)$$

La función f es la **relación verdadera** (desconocida en la práctica) y ϵ es el **ruido**: todo lo que influye en y y no medimos. El objetivo del aprendizaje es construir un modelo $h(x)$ (la **hipótesis**) lo más cercano posible a f , **no** a los puntos individuales, porque estos incluyen ruido.

En el notebook se usan datos sintéticos, de modo que sí conocemos la verdad:

$$\text{consumo}(v) = 5.2 + 0.0011 \cdot (v - 70)^2 + 0.000008 \cdot (v - 70)^3 \quad (1.2)$$

NOTA • Lectura de la función verdadera

El término cuadrático $(v - 70)^2$ produce la forma de **U** con mínimo cerca de 70 km/h. El pequeño término cúbico hace que la U sea **asimétrica**: el consumo crece más rápido a alta velocidad (resistencia del aire) que a baja velocidad (marchas cortas).

Como el ruido tiene $\sigma = 0.5$ L/100 km, **ningún modelo puede lograr, en promedio, un RMSE menor que 0.5 con datos nuevos**. Ese es el piso del error, llamado **error irreducible**.

1.2 La correlación de Pearson y sus límites

El coeficiente de correlación de Pearson mide la fuerza de la asociación **lineal** entre dos variables:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{[\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2]}} \quad (1.3)$$

Toma valores entre -1 y 1 . Vale ± 1 solo si los puntos están exactamente sobre una recta. En el notebook se obtiene $r = 0.707$, un valor apenas moderado, aunque visualmente la relación entre velocidad y consumo es **muy fuerte**. La razón: los puntos siguen una curva, y Pearson solo "ve" su tendencia general a subir.

ADVERTENCIA • Graficar siempre antes de modelar

Una correlación baja **no** significa que no exista relación; puede significar que la relación no es lineal. Una U perfectamente simétrica podría tener $r \approx 0$ y, sin embargo, una dependencia total entre las variables. La forma del diagrama de dispersión es la que decide el tipo de modelo.

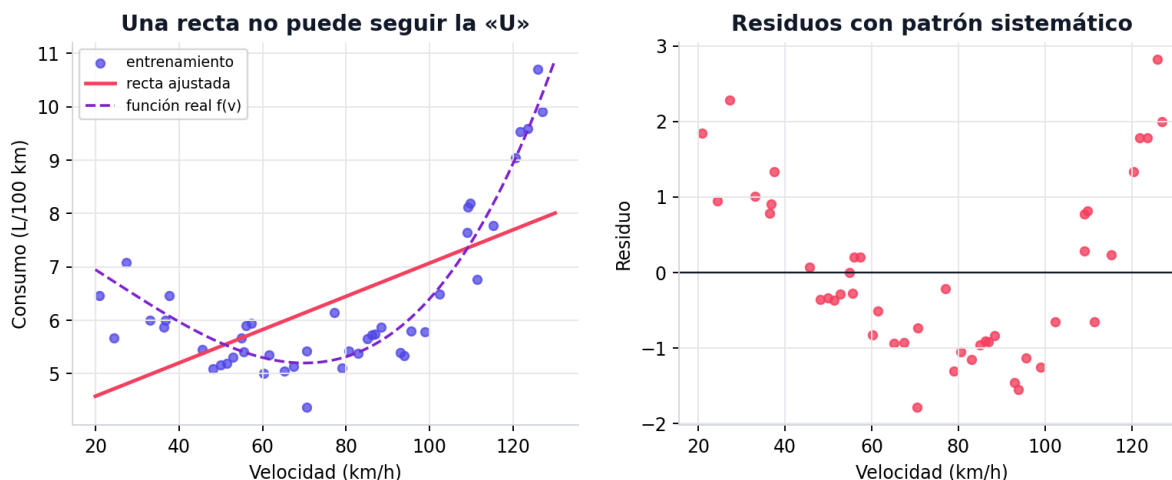


Figura 1. A la izquierda, la recta ajustada no puede seguir la U. A la derecha, sus residuos forman un patrón sistemático: el modelo subestima en los extremos y sobreestima en el centro.

La Figura 1 resume el diagnóstico del modelo base del notebook ($\hat{y} = 3.953 + 0.0312 \cdot v$, $R^2 = 0.405$ en entrenamiento). El patrón de los residuos es la señal de que falta estructura en el modelo; la sección 8 profundiza en esta herramienta.

02 El modelo de regresión polinomial

2.1 La ecuación

La regresión polinomial de grado d amplía la recta agregando potencias de la variable de entrada:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_d x^d \quad (2.1)$$

Con $d = 1$ se recupera la recta; con $d = 2$ se obtiene una parábola (una "U" o una "n"); con $d = 3$ una curva que puede tener dos cambios de dirección, y así sucesivamente. Un polinomio de grado d puede tener hasta $d - 1$ puntos de giro, lo que da una idea de su **flexibilidad**.

2.2 Lineal en los parámetros

DEFINICIÓN · Modelo lineal en los parámetros

Un modelo es **lineal en los parámetros** cuando $\hat{y}$ es una combinación lineal de los coeficientes β_j : cada coeficiente aparece multiplicando a una cantidad que **no depende de ningún otro coeficiente**, sin estar elevado a potencias ni dentro de funciones.

La regresión polinomial es **no lineal en x** (la curva no es recta) pero **lineal en β** . Esta es la idea central de todo el tema: por eso se resuelve con las mismas herramientas de la regresión lineal.

Modelo	¿Lineal en los parámetros?	Motivo
$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$	Sí	cada β multiplica a una potencia fija de x
$\hat{y} = \beta_0 + \beta_1 \cdot \ln x + \beta_2 \cdot \sqrt{x}$	Sí	$\ln x$ y $\sqrt{x}$ son solo nuevas columnas
$\hat{y} = \beta_0 \cdot e^{\beta_1 x}$	No	β_1 está dentro de la exponencial
$\hat{y} = \beta_0 + x^{\beta_1}$	No	β_1 actúa como exponente

2.3 El truco: crear nuevas características

Si definimos nuevas variables $z_1 = x$, $z_2 = x^2$, ..., $z_d = x^d$, la ecuación (2.1) se convierte en una **regresión lineal múltiple** ordinaria:

$$\hat{y} = \beta_0 + \beta_1 z_1 + \beta_2 z_2 + \dots + \beta_d z_d \quad (2.2)$$

Esto es exactamente lo que hace **PolynomialFeatures**: transforma la columna x en las columnas x , x^2 , ..., x^d . Después, **LinearRegression** trabaja sin saber que esas columnas son potencias. Organizando las n observaciones en una matriz, el modelo completo se escribe:

$$\begin{array}{l} \hat{y} = X \beta \\ \text{fila } i \text{ de } X: \quad [\quad 1 \quad x_i \quad x_i^2 \quad \dots \quad x_i^d \quad] \end{array} \quad (2.3)$$

La matriz X se llama **matriz de diseño**; en el caso polinomial recibe el nombre de **matriz de Vandermonde**. Con las tres velocidades del notebook y grado 3:

1 (sesgo)	v	v ²	v ³
1	20	400	8 000
1	70	4 900	343 000
1	130	16 900	2 197 000

NOTA • `include_bias=False`

La primera columna de unos corresponde al intercepto β_0 . Se omite en `PolynomialFeatures(include_bias=False)` porque `LinearRegression` ya calcula el intercepto por su cuenta; incluirla duplicaría esa columna.

2.4 Más de una variable de entrada

Con p variables, `PolynomialFeatures` genera todas las potencias y los productos cruzados (interacciones) hasta el grado d . Por ejemplo, con x_1 , x_2 y $d = 2$ se obtienen 1 , x_1 , x_2 , x_1^2 , x_1x_2 , x_2^2 . El número total de términos, incluido el sesgo, es:

$$\text{número de términos} = C(p + d, d) = (p + d)! / (p! \cdot d!) \quad (2.4)$$

Variables p	Grado 2	Grado 3	Grado 5
1	3	4	6
2	6	10	21
5	21	56	252
10	66	286	3 003

EN MACHINE LEARNING • La explosión combinatoria

El número de columnas crece muy rápido con p y d . Con muchas variables, un grado alto produce más características que observaciones, lo que garantiza el sobreajuste. Por eso en problemas multivariantes se suelen usar grados 2 o 3, o bien regularización (sección 11).

03 Estimación por mínimos cuadrados

3.1 Residuos y función de costo

Para cada observación, el **residuo** es la diferencia entre el valor real y el predicho, $e_i = y_i - \hat{y}_i$. Un buen modelo tiene residuos pequeños. El método de **mínimos cuadrados ordinarios** (MCO) elige los coeficientes que minimizan la suma de los residuos al cuadrado:

$$\begin{aligned} J(\beta) &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n e_i^2 \\ J(\beta) &= (y - X\beta)^T (y - X\beta) \quad (\text{forma matricial}) \end{aligned} \quad (3.1)$$

3.2 Las ecuaciones normales

$J(\beta)$ es una función cuadrática y convexa de β , con forma de "cuenca". Su mínimo está donde el gradiente se anula. Derivando respecto de β :

$$\begin{aligned}\nabla J(\beta) &= -2 X^T(y - X\beta) = 0 \\ X^T X \beta &= X^T y \quad (\text{ecuaciones normales}) \\ \beta^* &= (X^T X)^{-1} X^T y \quad (3.2)\end{aligned}$$

Esta es la misma fórmula de la regresión lineal múltiple: el carácter polinomial está únicamente dentro de la matriz X . Hay una solución única siempre que las columnas de X sean linealmente independientes, lo que exige al menos $d + 1$ valores distintos de x .

3.3 Ejemplo resuelto a mano

EJEMPLO · Ajustar una parábola a cuatro puntos

Datos: $x = (0, 1, 2, 3)$ e $y = (3, 1, 2.5, 4.5)$. Se busca $\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2$.

Paso 1. Matriz de diseño con columnas $1, x, x^2$: filas $(1, 0, 0), (1, 1, 1), (1, 2, 4), (1, 3, 9)$.

Paso 2. Se calculan los productos que forman las ecuaciones normales:

$$\begin{aligned}X^T X &= \begin{bmatrix} 4 & 6 & 14 \\ 6 & 14 & 36 \\ 14 & 36 & 98 \end{bmatrix} \\ X^T y &= \begin{bmatrix} 11 & 19.5 & 51.5 \end{bmatrix}\end{aligned}$$

Paso 3. Resolviendo el sistema de 3 ecuaciones: $\beta^*_0 = 2.85, \beta^*_1 = -2.4, \beta^*_2 = 1.0$, es decir:

$$\hat{y} = 2.85 - 2.4x + x^2$$

Paso 4. Predicciones $(2.85, 1.45, 2.05, 4.65)$; residuos $(0.15, -0.45, 0.45, -0.15)$; suma de cuadrados de los residuos = 0.45. Ninguna otra parábola logra una suma menor.

In [1]:

```
import numpy as np
x = np.array([0, 1, 2, 3]); y = np.array([3, 1, 2.5, 4.5])
X = np.column_stack([np.ones_like(x), x, x**2])
beta = np.linalg.solve(X.T @ X, X.T @ y)
print(beta)
print(np.polyfit(x, y, deg=2)) # mismo resultado, de mayor a menor grado
```

Out [1]:

```
[ 2.85 -2.4  1.  ]
[ 1.   -2.4  2.85]
```

3.4 Propiedades útiles de la solución

- ▶ **Los residuos suman cero** cuando el modelo tiene intercepto: en el ejemplo, $0.15 - 0.45 + 0.45 - 0.15 = 0$. Por eso la media de los residuos en el notebook es -0.0000 .
- ▶ **Los residuos son ortogonales a cada columna de X :** $X^T e = 0$. Geométricamente, $\hat{y}$ es la proyección de y sobre el espacio generado por las columnas de X .
- ▶ **¿Por qué cuadrados y no valores absolutos?** El cuadrado es derivable (da una solución cerrada), penaliza más los errores grandes y, si el ruido es normal, MCO coincide con el estimador de **máxima verosimilitud**.

NOTA • En la práctica no se invierte la matriz

Calcular $(X^T X)^{-1}$ explícitamente es numéricamente inestable. Las bibliotecas (`np.linalg.lstsq`, `LinearRegression`) usan descomposiciones como QR o SVD, que llegan al mismo β^* con más precisión. La fórmula (3.2) es para entender, no para programar.

04 Escalas, multicolinealidad y estandarización

4.1 El problema de las escalas

En la tabla de la sección 2.3, v llega a 130 mientras que v^3 supera los dos millones. Columnas con órdenes de magnitud tan distintos causan dos problemas: coeficientes de tamaños absurdamente diferentes y errores de redondeo al resolver las ecuaciones normales. Con grado 10, v^{10} llega a 1.4×10^{21} .

4.2 Multicolinealidad

Además, las potencias de una variable positiva "se mueven juntas": si v crece, también crecen v^2 y v^3 . En los datos de entrenamiento del notebook:

	v	v^2	v^3
v	1.000	0.982	0.946
v^2	0.982	1.000	0.989
v^3	0.946	0.989	1.000

DEFINICIÓN • Multicolinealidad

Situación en la que dos o más columnas de X están fuertemente correlacionadas. La matriz $X^T X$ queda casi singular (casi no invertible): pequeños cambios en los datos producen grandes cambios en los coeficientes β_j^* . Las **predicciones** pueden seguir siendo buenas, pero los **coeficientes individuales** se vuelven inestables e ininterpretables.

Una forma de medir el problema es el **número de condición** κ de la matriz de diseño: cuanto más grande, más sensible es la solución a errores numéricos. Para los datos de entrenamiento con grado 3:

Matriz de diseño (grado 3)	Número de condición κ
Columnas originales 1, v , v^2 , v^3	$\approx 11\,800\,000$
Columnas estandarizadas	≈ 91

4.3 Estandarización

`StandardScaler` transforma cada columna para que tenga media 0 y desviación estándar 1:

$$z = (x - \mu) / s \quad (4.1)$$

donde μ y s son la media y la desviación estándar de la columna **calculadas con los datos de entrenamiento**. Tras estandarizar, todas las columnas tienen escalas comparables y el número de condición baja cinco órdenes de magnitud.

4.4 Pipeline y fuga de datos

El orden correcto es: generar potencias → estandarizar → ajustar. Un **Pipeline** encadena los tres pasos y garantiza que **fit** aprenda μ y s **solo** con los datos que se le entregan para entrenar.

In [2]:

```
def modelo_polinomial(grado):
    return make_pipeline(
        PolynomialFeatures(degree=grado, include_bias=False),
        StandardScaler(),
        LinearRegression(),
    )
```

ADVERTENCIA • Fuga de datos (data leakage)

Si se estandariza el conjunto completo **antes** de dividir en entrenamiento y prueba, las medias y desviaciones incluyen información de los datos de prueba. El modelo "espía" el examen y la evaluación resulta optimista.

Dentro de la validación cruzada ocurre lo mismo con cada pliegue: el **Pipeline** reajusta el escalador en cada iteración usando solo los pliegues de ajuste.

ADVERTENCIA • Coeficientes estandarizados

Los coeficientes del modelo del Pipeline están expresados en la escala estandarizada de v , v^2 , v^3 , por lo que no se pueden leer como "litros por km/h". Para interpretar el modelo en unidades originales se usa **np.polyfit** (sección 9).

05 Métricas de evaluación

Una vez ajustado el modelo, hay que cuantificar qué tan buenas son sus predicciones. El notebook usa tres métricas complementarias:

$$\begin{aligned} \text{MSE} &= (1/n) \sum (y_i - \hat{y}_i)^2 \\ \text{RMSE} &= \sqrt{\text{MSE}} \\ \text{MAE} &= (1/n) \sum |y_i - \hat{y}_i| \end{aligned} \quad (5.1)$$

$$\begin{aligned} R^2 &= 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}} \\ \text{SS}_{\text{res}} &= \sum (y_i - \hat{y}_i)^2 \quad \text{SS}_{\text{tot}} = \sum (y_i - \bar{y})^2 \end{aligned} \quad (5.2)$$

Métrica	Unidades	Interpretación	Mejor valor
RMSE	las de y (L/100 km)	error "típico"; penaliza más los errores grandes	0

Métrica	Unidades	Interpretación	Mejor valor
MAE	las de y	error promedio en valor absoluto; robusto a extremos	0
R^2	sin unidades	fracción de la variabilidad de y explicada	1

NOTA • Cómo leer R^2

R^2 compara el modelo con el predictor más ingenuo: predecir siempre la media $\bar{y}$. Si $R^2 = 0$, el modelo no mejora esa referencia; si es negativo, es **peor** que predecir la media (ocurre en prueba con modelos sobreajustados).

Siempre se cumple $RMSE \geq MAE$. Si el RMSE es mucho mayor que el MAE, hay algunos errores muy grandes.

Resultados en el conjunto de prueba del notebook:

Modelo	RMSE	MAE	R^2
Lineal (grado 1)	1.359	1.182	0.522
Polinomial (grado 3)	0.476	0.381	0.941

EN MACHINE LEARNING • El ruido fija un piso

Con $\sigma = 0.5$, el RMSE esperado del **modelo perfecto** ($h = f$) con datos nuevos es ≈ 0.5 . El grado 3 obtiene 0.476 en prueba: está en el piso, y la pequeña diferencia se debe al azar de solo 15 observaciones.

En entrenamiento el RMSE puede quedar **por debajo** de 0.5 (0.42 con grado 3) porque el ajuste aprovecha parte del ruido de esos mismos puntos. Un RMSE de prueba muy inferior al nivel del ruido sería una señal de error en el procedimiento, por ejemplo una fuga de datos.

06 Sesgo, varianza y el grado del polinomio

El grado d es un **hiperparámetro**: no se aprende con mínimos cuadrados, sino que lo elige quien construye el modelo. Controla la **complejidad** del modelo y, con ella, dos tipos de error opuestos.

6.1 Subajuste y sobreajuste

DEFINICIÓN • Subajuste (underfitting) y sobreajuste (overfitting)

Subajuste: el modelo es demasiado simple para la forma de los datos. Tiene error alto tanto en entrenamiento como en prueba. Se asocia a **alto sesgo**.

Sobreajuste: el modelo es tan flexible que aprende el ruido de los puntos de entrenamiento. Tiene error bajo en entrenamiento y alto en prueba. Se asocia a **alta varianza**.

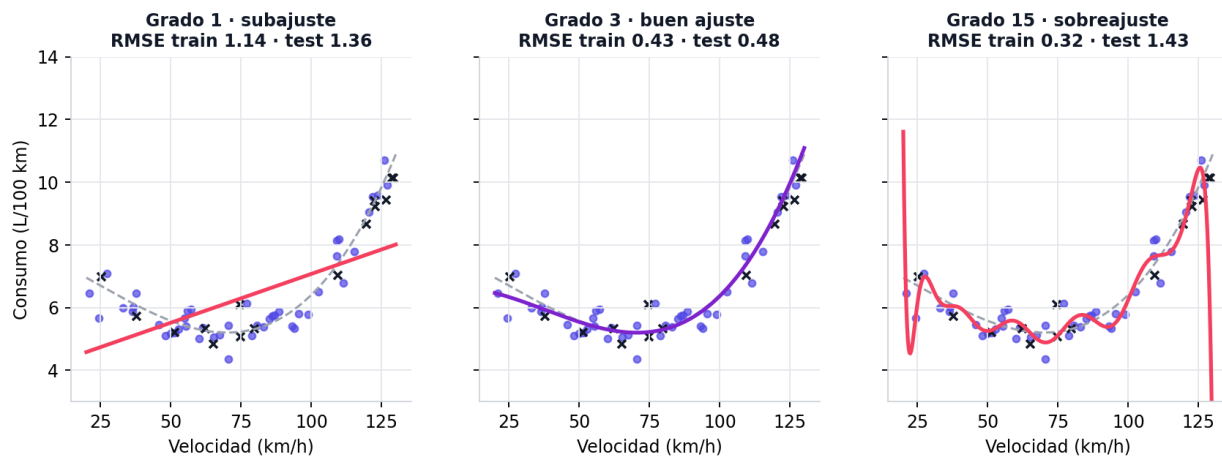


Figura 2. Tres grados sobre los mismos datos (círculos: entrenamiento; cruces: prueba; línea gris: función real). El grado 15 persigue puntos individuales y se dispara en los extremos.

6.2 La descomposición sesgo-varianza

Imaginemos repetir el experimento muchas veces con muestras distintas y ajustar un modelo en cada una. El error esperado al predecir en un punto x se descompone en tres términos:

$$E[(y - h(x))^2] = \text{Sesgo}[h(x)]^2 + \text{Var}[h(x)] + \sigma^2 \quad (6.1)$$

- **Sesgo:** cuánto se aleja, en promedio, la curva ajustada de la curva verdadera. Una recta nunca podrá imitar una U, sin importar cuántos datos tenga: sesgo alto.
- **Varianza:** cuánto cambia la curva ajustada de una muestra a otra. Un polinomio de grado 15 cambia radicalmente si se reemplazan unos pocos puntos: varianza alta.
- σ^2 : el ruido irreducible. Ningún modelo lo elimina.

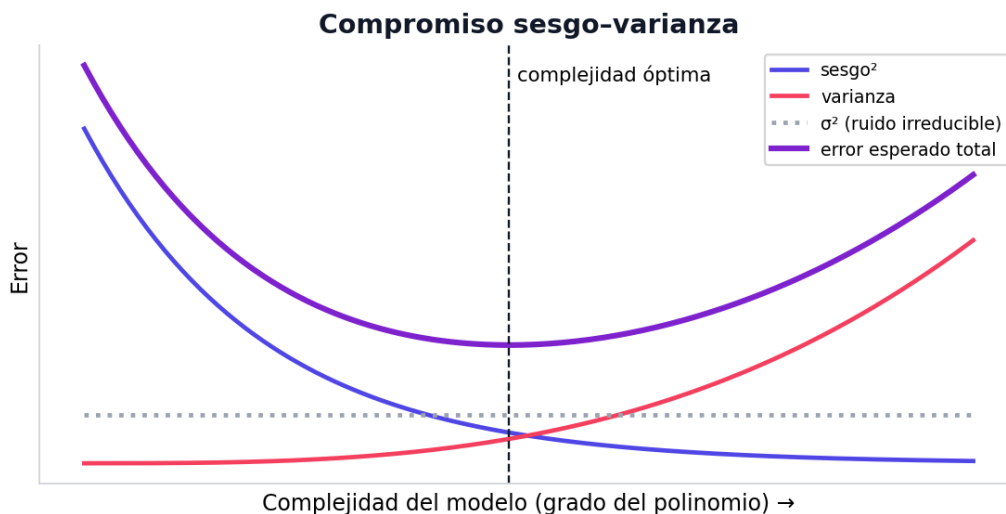


Figura 3. Al aumentar la complejidad, el sesgo baja y la varianza sube. El error total tiene un mínimo en una complejidad intermedia.

6.3 ¿Por qué el error de entrenamiento siempre baja?

Los polinomios forman **modelos anidados**: un polinomio de grado 3 es un caso particular del de grado 4 (con $\beta_4 = 0$). Como mínimos cuadrados busca el mejor ajuste posible dentro de cada familia, el de grado 4 puede hacerlo al menos tan bien como el de grado 3 en los mismos datos. En consecuencia, **el error de entrenamiento nunca aumenta al subir el grado**; con $d = n - 1$ la curva pasa exactamente por todos los puntos y el error de entrenamiento es cero.

ADVERTENCIA · Consecuencia práctica

El error de entrenamiento **no sirve** para elegir el grado: siempre favorecerá el grado más alto. Se necesita un error medido con datos que el modelo no haya visto (sección 7).

6.4 El fenómeno de Runge

Los polinomios de grado alto tienden a **oscilar violentamente cerca de los extremos** del intervalo de datos, aunque se ajusten bien en el centro. Carl Runge lo describió en 1901: aumentar el grado de un polinomio interpolador no siempre mejora la aproximación, y en los bordes puede empeorarla. Es exactamente lo que se observa con el grado 15 en la Figura 2 y lo que hace tan peligrosa la extrapolación (sección 10).

07 Validación cruzada y selección del grado

7.1 Tres papeles para los datos

Conjunto	Para qué se usa	Cuándo
Entrenamiento	ajustar los coeficientes β	siempre
Validación	comparar modelos y elegir hiperparámetros (el grado)	durante el diseño
Prueba	estimar el rendimiento final con datos nunca vistos	una sola vez, al final

En el notebook, 75 % de los datos (45) se destinan a entrenamiento y 25 % (15) a prueba. El conjunto de validación no se separa de forma fija: se obtiene repetidamente **dentro** del de entrenamiento mediante validación cruzada.

ADVERTENCIA · Usar la prueba para decidir es una fuga de datos

Si se elige el grado mirando el error de prueba, esa decisión se "ajusta" a esas 15 observaciones concretas y la estimación final deja de ser honesta. El conjunto de prueba se guarda "bajo llave".

7.2 Validación cruzada k-fold

El conjunto de entrenamiento se divide en k partes (pliegues o *folds*) de tamaño similar. Se entrena k veces: en cada iteración se usa un pliegue distinto para validar y los $k - 1$ restantes para ajustar. El error de validación cruzada es el promedio:

$$CV_k = (1/k) \sum_{j=1}^k RMSE_j \quad (7.1)$$

Validación cruzada k-fold (k = 5) sobre el conjunto de ENTRENAMIENTO

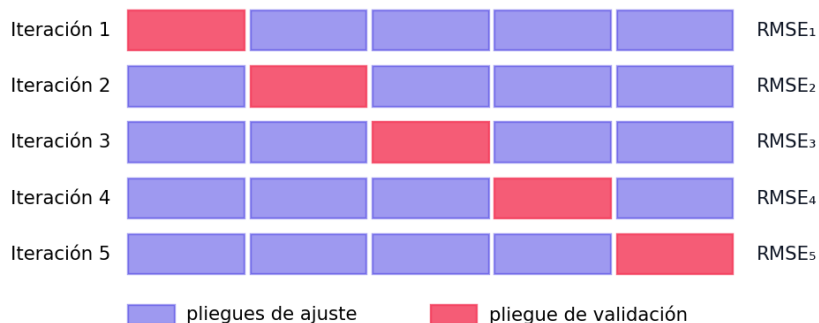


Figura 4. Esquema de validación cruzada con k = 5. Cada observación se usa exactamente una vez para validar.

NOTA • ¿Por qué shuffle=True?

El DataFrame del notebook está ordenado por velocidad. Sin mezclar, cada pliegue contendría un rango contiguo de velocidades y el modelo tendría que **extrapolar** a ese rango en cada iteración, inflando el error. Mezclar hace que cada pliegue sea representativo del conjunto.

7.3 La curva de validación

Repetiendo la validación cruzada para cada grado de 1 a 15 se obtiene la **curva de validación**, el gráfico más importante del notebook:

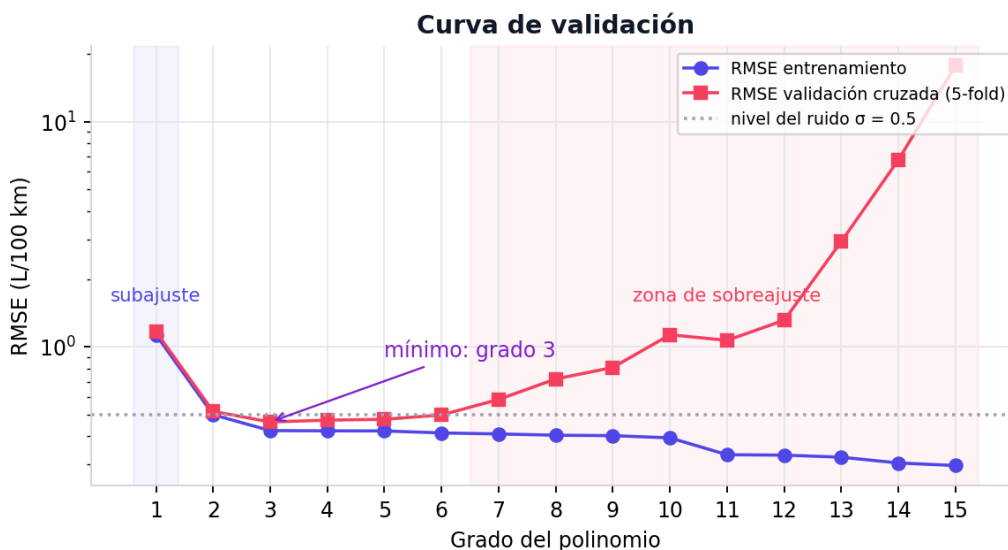


Figura 5. Curva de validación (escala logarítmica). El RMSE de entrenamiento baja siempre; el de validación baja, alcanza un mínimo en grado 3 y luego sube.

Grado	RMSE entrenamiento	RMSE validación	Desv. validación
1	1.130	1.166	0.173
2	0.499	0.516	0.068
3	0.424	0.463	0.035

Grado	RMSE entrenamiento	RMSE validación	Desv. validación
4	0.423	0.472	0.034
5	0.422	0.476	0.034
6	0.414	0.498	0.048
8	0.404	0.719	0.379
12	0.330	1.312	1.139
15	0.297	17.781	23.626

Lectura: la **distancia** entre las dos curvas mide el sobreajuste. Observe también que la desviación de la validación se dispara con grados altos: el modelo cambia mucho de un pliegue a otro, que es precisamente la definición de **varianza alta**.

7.4 La regla de un error estándar

Cuando varios grados tienen errores de validación muy parecidos, las diferencias pueden deberse al azar. El principio de **parsimonia** (navaja de Ockham) recomienda preferir el modelo más simple. La regla de un error estándar lo formaliza:

$$\begin{aligned}
 SE &= s_{\text{val}} / \sqrt{k} \\
 \text{umbral} &= \text{RMSE}_{\text{mín}} + SE \\
 \text{elegir el menor grado con } \text{RMSE}_{\text{val}} &\leq \text{umbral} \quad (7.2)
 \end{aligned}$$

EJEMPLO • Aplicación con los datos del notebook

Mínimo en grado 3: RMSE = 0.463 y $s = 0.035$. Con $k = 5$: $SE = 0.035 / \sqrt{5} \approx 0.0157$, $\text{umbral} = 0.463 + 0.0157 \approx 0.479$. El grado 2 (0.516) queda fuera; el grado más simple dentro del umbral es **3**. En este caso ambos criterios coinciden, pero no siempre sucede.

7.5 Automatizar con GridSearchCV

In [3]:

```

busqueda = GridSearchCV(modelo_polinomial(1),
    param_grid={"polynomialfeatures__degree": range(1, 16)},
    cv=kf, scoring="neg_root_mean_squared_error")
busqueda.fit(X_train, y_train)
busqueda.best_params_

```

Out [3]:

```
{'polynomialfeatures__degree': 3}
```

- El nombre `polynomialfeatures__degree` sigue la convención `paso__parámetro` de los pipelines (doble guion bajo).

- ▶ scikit-learn siempre **maximiza** la puntuación; por eso el RMSE se expresa como `neg_root_mean_squared_error` (su valor negativo) y se cambia el signo al reportarlo.
- ▶ Con `refit=True` (valor por defecto), `GridSearchCV` reentrena el mejor modelo con **todo** el conjunto de entrenamiento.

08 Análisis de residuos

Las métricas resumen el error en un solo número; los residuos muestran **dónde** y **cómo** se equivoca el modelo. El modelo de regresión se apoya en supuestos que se verifican con gráficos de residuos:

Supuesto	Qué significa	Cómo se revisa	Señal de problema
Forma funcional correcta	el modelo captura la tendencia	residuos vs. predicción o vs. x	curvas o patrones (U, ondas)
Homocedasticidad	varianza del error constante	residuos vs. predicción	forma de embudo
Normalidad	errores ~ normales, media 0	histograma o gráfico Q-Q	asimetría, colas pesadas
Independencia	errores no relacionados entre sí	residuos en el orden de captura	rachas o ciclos

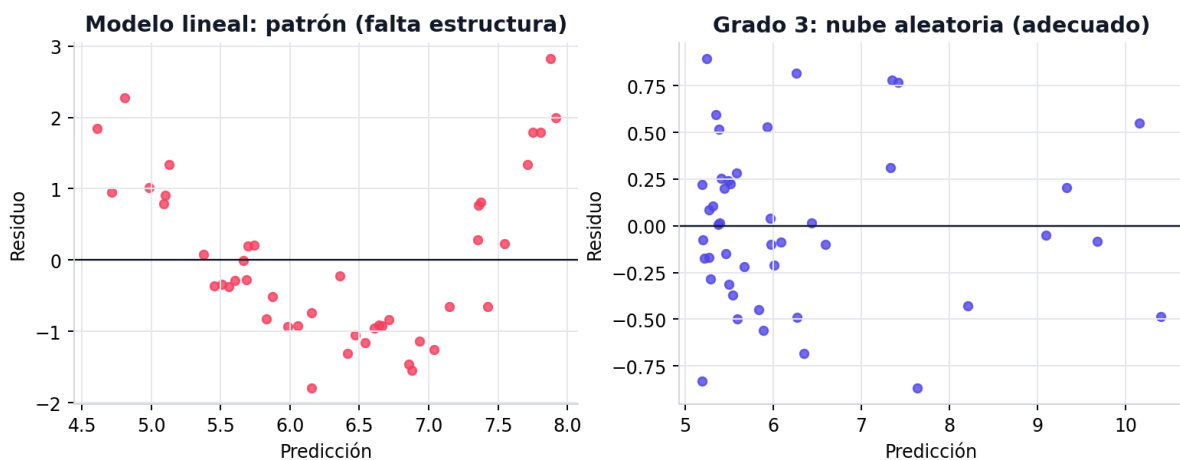


Figura 6. Residuos del modelo lineal (patrón en U: falta estructura) frente a los del grado 3 (nube aleatoria alrededor de cero).

En el modelo final del notebook la media de los residuos es 0 (propiedad de la sección 3.4) y su desviación es 0.428, próxima al ruido real de 0.5. Se concluye que el modelo capturó toda la estructura recuperable de los datos.

EN MACHINE LEARNING • Diagnóstico con residuos

Un patrón en los residuos es la señal para **aumentar** la complejidad (agregar términos). Residuos aleatorios con error de prueba mucho mayor que el de entrenamiento indican lo contrario: hay que **reducir** la complejidad o regularizar.

09 Interpretación mediante derivadas

En la regresión lineal simple, β_1 tiene una lectura directa: el cambio de y por cada unidad adicional de x . En un polinomio esto **ya no es así**: el efecto de x depende del punto donde se esté. La herramienta correcta es la **derivada**.

9.1 El efecto marginal

Para un polinomio de grado 3, $p(v) = a_3v^3 + a_2v^2 + a_1v + a_0$, la pendiente en cada velocidad es:

$$\begin{aligned}p'(v) &= 3a_3v^2 + 2a_2v + a_1 \\ p''(v) &= 6a_3v + 2a_2\end{aligned}\quad (9.1)$$

$p'(v)$ indica cuántos L/100 km cambia el consumo por cada km/h adicional **a esa velocidad**. Es el **efecto marginal**. Con los coeficientes del notebook, obtenidos con `np.polyfit`:

$$p(v) = 1.062 \times 10^{-5}v^3 - 0.001214v^2 + 0.01274v + 6.61 \quad (9.2)$$

Velocidad	Pendiente $p'(v)$	Interpretación
40 km/h	-0.0334	acelerar reduce el consumo
70 km/h	-0.0010	prácticamente plano: cerca del óptimo
110 km/h	+0.1313	cada km/h adicional aumenta el consumo

ADVERTENCIA • No interpretar coeficientes aislados

Por la multicolinealidad (sección 4.2), los coeficientes individuales de un polinomio son inestables y no tienen significado propio. Un modelo polinomial se interpreta mediante **la curva y sus derivadas**, nunca leyendo cada β_j por separado. Recuerde también que `np.polyfit` devuelve los coeficientes **de mayor a menor grado**.

9.2 El punto óptimo

El consumo es mínimo donde la pendiente se anula: $p'(v) = 0$. Es una ecuación de segundo grado con dos soluciones:

$$\begin{aligned}3.186 \times 10^{-5}v^2 - 0.002428v + 0.01274 &= 0 \\ v \approx 70.5 \text{ km/h} \quad \text{y} \quad v \approx 5.7 \text{ km/h}\end{aligned}\quad (9.3)$$

El criterio de la segunda derivada decide su naturaleza: en $v \approx 70.5$, $p'' > 0$, por lo que es un **mínimo**; en $v \approx 5.7$, $p'' < 0$, un máximo. Además, 5.7 km/h está **fuera** del rango de los datos (20–130), así que no tiene significado físico: es un artefacto del polinomio. El notebook busca el mínimo solo dentro del rango observado y obtiene 70.5 km/h, muy cerca del óptimo real (70 km/h).

Caso particular, la parábola: para $\hat{y} = \beta_0 + \beta_1x + \beta_2x^2$ el vértice está en $x^* = -\beta_1/(2\beta_2)$; es mínimo si $\beta_2 > 0$ y máximo si $\beta_2 < 0$. En el ejemplo de la sección 3.3, $x^* = 1.2$.

10 El peligro de extrapolar

DEFINICIÓN · Interpolar y extrapolar

Interpolar es predecir dentro del rango de valores de x observados en el entrenamiento. **Extrapolar** es predecir fuera de ese rango.

Fuera del rango de los datos, el término de mayor grado $\beta_d x^d$ domina a todos los demás y la curva tiende a $\pm\infty$ a una velocidad que crece con d . El modelo no tiene ningún dato que lo "sujete" en esa zona, de modo que su comportamiento depende de detalles numéricos sin relación con el fenómeno.

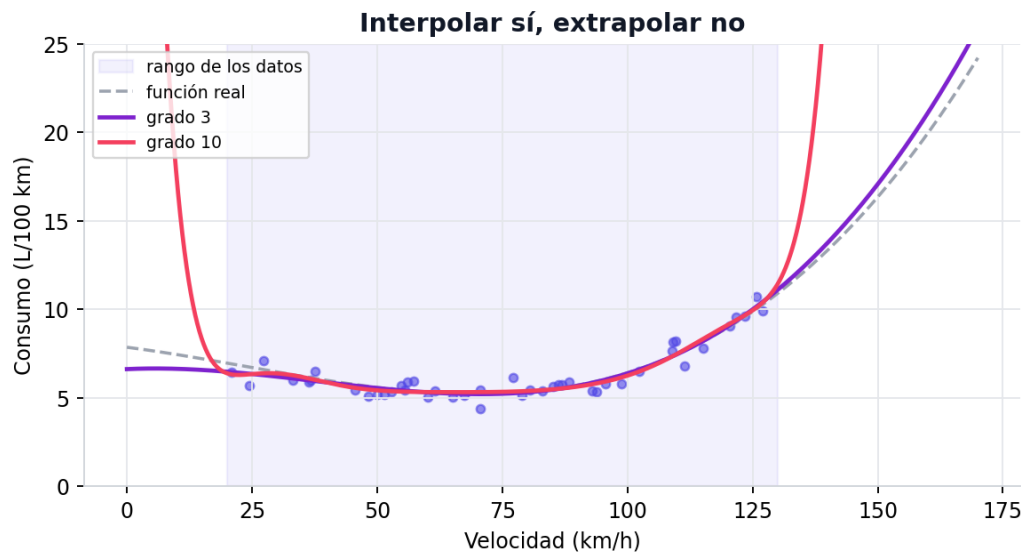


Figura 7. Dentro de la franja sombreada (20–130 km/h) ambos modelos son razonables; fuera de ella el grado 10 se dispara.

Predicción a 160 km/h	Consumo (L/100 km)
Función real	19.94
Polinomio de grado 3	21.09
Polinomio de grado 10	164.47 (valor absurdo)

Incluso el grado 3, que es el modelo correcto en su forma, pierde precisión al alejarse del rango. El valor exacto del grado 10 puede variar de una versión de las bibliotecas a otra: esa misma inestabilidad es la prueba de que la predicción no es confiable.

ADVERTENCIA · Regla

La regresión polinomial sirve para **interpolar**. Si el problema exige predecir fuera del rango observado, se necesitan más datos en esa zona o un modelo basado en el conocimiento del fenómeno (por ejemplo, un modelo físico de resistencia aerodinámica).

11 Regularización Ridge

Elegir el grado es una forma "discreta" de controlar la complejidad. La **regularización** ofrece una alternativa continua: permite usar un grado alto pero **penaliza los coeficientes grandes**, que son los que producen oscilaciones. Ridge (regularización L2) modifica la función de costo:

$$\begin{aligned} J_{\text{Ridge}}(\beta) &= \sum (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^d \beta_j^2 \\ \beta_{\text{Ridge}}^* &= (X^T X + \alpha I)^{-1} X^T y \end{aligned} \quad (11.1)$$

- ▶ $\alpha \geq 0$ controla la intensidad de la penalización. Con $\alpha = 0$ se recupera mínimos cuadrados; con α muy grande los coeficientes tienden a 0 y el modelo se vuelve casi constante (subajuste).
- ▶ Sumar αI a la diagonal hace que la matriz sea siempre invertible y bien condicionada: Ridge también **remedia la multicolinealidad**.
- ▶ El intercepto β_0 no se penaliza (la suma empieza en $j = 1$).
- ▶ La penalización depende de la escala de cada columna, por lo que **es obligatorio estandarizar** antes de aplicar Ridge.
- ▶ α es otro hiperparámetro y se elige con validación cruzada, igual que el grado.

EN MACHINE LEARNING · Ridge, Lasso y Elastic Net

Ridge encoge todos los coeficientes pero rara vez los hace exactamente cero. **Lasso** (penalización L1, $\alpha \sum |\beta_j|$) puede anular coeficientes y así "elegir" qué potencias conservar. **Elastic Net** combina ambas. Son la base de la regularización que reaparece en redes neuronales como *weight decay*.

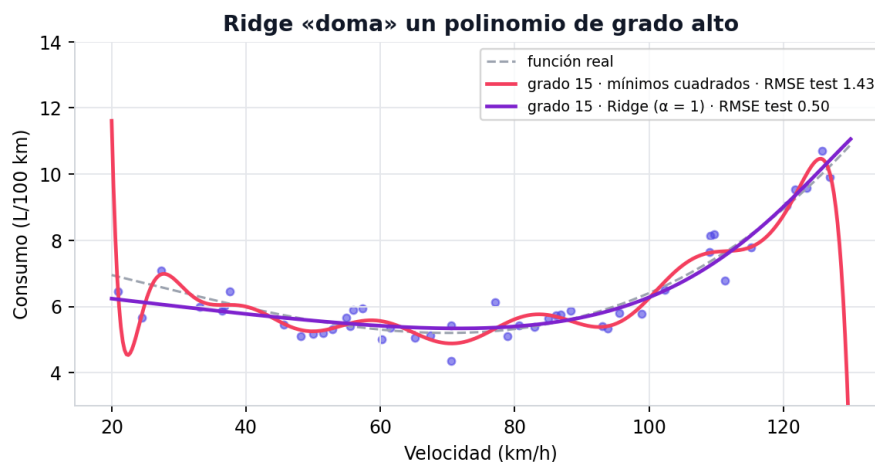


Figura 8. Un polinomio de grado 15 ajustado por mínimos cuadrados y por Ridge ($\alpha = 1$) sobre los mismos datos de entrenamiento.

La diferencia se explica por el tamaño de los coeficientes. En este experimento, la norma del vector de coeficientes estandarizados del grado 15 es del orden de 10^{11} con mínimos cuadrados y de apenas 1.2 con Ridge: la penalización obliga al modelo a repartir el efecto entre las columnas en lugar de compensar coeficientes gigantes de signos opuestos.

In [4]:

```
from sklearn.linear_model import Ridge
ridge15 = make_pipeline(PolynomialFeatures(15, include_bias=False),
                        StandardScaler(), Ridge(alpha=1.0))
ridge15.fit(X_train, y_train)
```

12 Resumen y flujo de trabajo

12.1 Flujo de trabajo recomendado

1. **Graficar** los datos: la forma de la nube decide el modelo; la correlación de Pearson no basta.
2. **Separar** entrenamiento y prueba antes de cualquier otra decisión.
3. Ajustar una **línea base** lineal y revisar sus residuos: un patrón indica que faltan términos.
4. Construir un **Pipeline**: `PolynomialFeatures` → `StandardScaler` → `LinearRegression` (o `Ridge`).
5. Elegir el grado (y α) con **validación cruzada** sobre el entrenamiento; aplicar la regla de parsimonia.
6. Evaluar **una sola vez** en prueba con RMSE, MAE y R^2 ; comparar con el nivel de ruido.
7. Revisar los **residuos** del modelo final.
8. **Interpretar** con la curva y sus derivadas, y **no extrapolar**.

12.2 Glosario

Término	Definición breve
Regresión polinomial	regresión lineal sobre las potencias $x, x^2, \dots, x^d$ de la entrada
Matriz de diseño / Vandermonde	matriz X con una fila por observación y columnas $1, x, \dots, x^d$
Mínimos cuadrados	método que minimiza la suma de los residuos al cuadrado
Ecuaciones normales	$X^T X \beta = X^T y$; su solución da los coeficientes óptimos
Multicolinealidad	columnas muy correlacionadas que vuelven inestables los coeficientes
Estandarización	transformar cada columna a media 0 y desviación 1
Hiperparámetro	valor que no se aprende de los datos (grado, α) y se elige con validación
Sesgo / varianza	error por rigidez del modelo / error por sensibilidad a la muestra
Validación cruzada k-fold	k entrenamientos rotando el pliegue de validación; se promedia el error
Fuga de datos	uso de información de validación o prueba durante el entrenamiento
Fenómeno de Runge	oscilaciones de los polinomios de grado alto en los extremos
Ridge	regresión con penalización $\alpha \sum \beta_j^2$ sobre los coeficientes